

Tuần 7

Phát hiện Cạnh & Liên kết Cạnh

Gradient · Canny · Non-max Suppression · Hysteresis · Contour

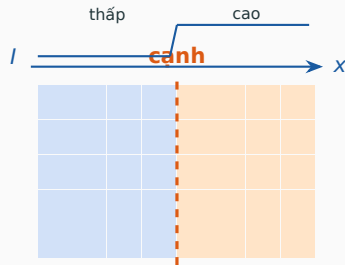
Xử lý ảnh và Thị giác máy tính · 121036

Máy nhìn thấy gì khi nhìn vào một vật thể?

Não người nhận diện vật thể qua **đường viền** và **hình dạng**, không phải qua từng giá trị pixel.

Câu hỏi cốt lõi:

Làm thế nào để máy tính tìm được đường viền đó?

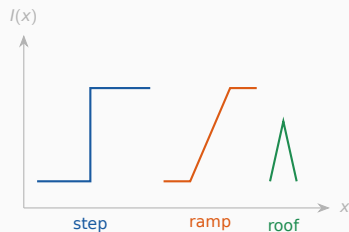


Cạnh là gì và tại sao ta cần nó?

Cạnh (edge) = nơi cường độ ảnh thay đổi đột ngột.

Cạnh mã hoá:

- Biên vật thể (object boundary)
- Thay đổi chiều sâu (depth discontinuity)
- Bóng và ánh sáng (shadow / shading)
- Kết cấu bề mặt (texture boundary)



Ứng dụng: phân đoạn ảnh, nhận dạng vật thể, ước lượng pose, đo lường 3D.

Building block: Đạo hàm ảnh

Nguyên lý: cạnh $\leftrightarrow$ đỉnh của $|\nabla I|$ (gradient magnitude).

Đạo hàm rời rạc theo x:

$$\frac{\partial I}{\partial x} \approx I(x+1, y) - I(x, y)$$

Đạo hàm theo y:

$$\frac{\partial I}{\partial y} \approx I(x, y+1) - I(x, y)$$

Gradient vector:

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right)$$

Magnitude (độ mạnh cạnh):

$$M = \sqrt{\left(\frac{\partial I}{\partial x} \right)^2 + \left(\frac{\partial I}{\partial y} \right)^2}$$

Direction (hướng cạnh):

$$\theta = \arctan\left(\frac{\partial I / \partial y}{\partial I / \partial x} \right)$$

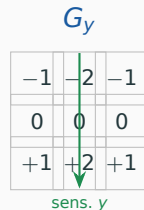
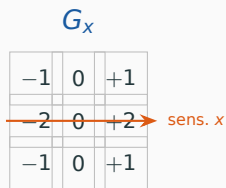
Lưu ý: θ vuông góc với đường cạnh thật.

Bộ lọc Sobel: tính gradient trên thực tế

Sobel = đạo hàm rời rạc + làm mịn Gaussian 1D theo hướng vuông góc.

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$



Tại mỗi pixel: tích chập G_x , $G_y \rightarrow$ tính M và θ .

Vấn đề: Gradient thô chưa đủ

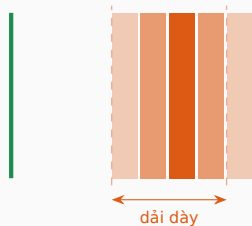
Sobel thuần cho ra **cạnh dày**
(nhiều pixel):

- × Một cạnh thật → dải 3-5 pixel
- × Nhiều ảnh → gradient giả
- × Không biết pixel nào là cạnh "chính xác"

Cần một pipeline hoàn chỉnh để:

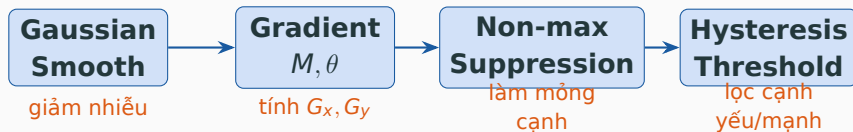
- (1) giảm nhiễu trước khi lấy gradient
- (2) làm mỏng cạnh còn 1 pixel
- (3) lọc cạnh yếu vs. cạnh mạnh

Cạnh lý tưởng Gradient Sobel



Canny Edge Detector: 4 bước

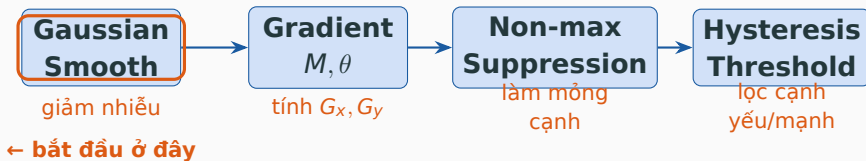
John Canny (1986) extendash tiêu chuẩn công nghiệp cho phát hiện cạnh.



Mỗi bước giải quyết *đúng một* vấn đề cụ thể. Ta sẽ xây từng bước.

Canny Edge Detector: 4 bước

John Canny (1986) extendash tiêu chuẩn công nghiệp cho phát hiện cạnh.



Mỗi bước giải quyết *đúng một vấn đề* cụ thể. Ta sẽ xây từng bước.

Bước 1 extendash Gaussian Smoothing

Vấn đề cần giải: nhiễu tạo ra gradient lớn giả.

Giải pháp: làm mờ ảnh bằng **Gaussian kernel** trước khi lấy đạo hàm.

$$I_{\sigma} = G_{\sigma} * I, \quad G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- σ nhỏ $\rightarrow$ giữ chi tiết, nhạy nhiễu
- σ lớn $\rightarrow$ mờ hơn, bỏ qua chi tiết nhỏ
- **Trade-off:** localisation vs. noise immunity



Bước 2 - Gradient Magnitude & Direction

Áp dụng Sobel lên **ảnh đã làm mờ** I_σ :

$$G_x = \frac{\partial I_\sigma}{\partial x}, \quad G_y = \frac{\partial I_\sigma}{\partial y}$$

$$\mathbf{M}(x, y) = \sqrt{G_x^2 + G_y^2}, \quad \theta(x, y) = \arctan(G_y/G_x)$$

M

θ

5	80	82	6
4	78	79	5
3	6	5	4

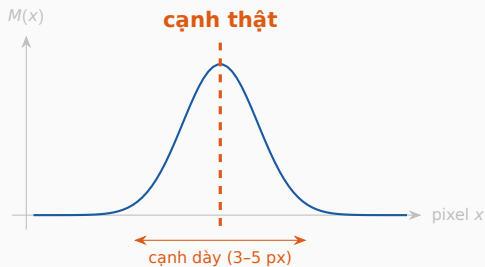
→
+ θ

—	90°	88°	—
—	91°	90°	—
—	—	—	—

Kết quả: hai ảnh M và θ – một pixel vẫn có thể là *cạnh dày*.

Tại sao cần Bước 3? - Vấn đề cạnh dày

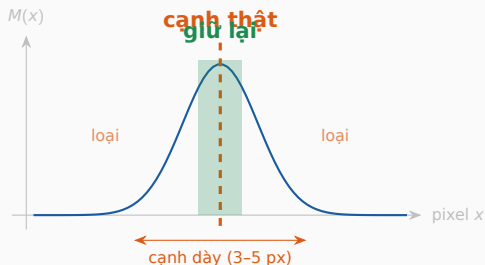
Sau bước 2, một cạnh thật thường chiếm **nhiều pixel liên tiếp** trong M .



Mục tiêu Bước 3: giữ *chỉ* pixel có M cực đại cực bộ theo hướng θ .

Tại sao cần Bước 3? - Vấn đề cạnh dày

Sau bước 2, một cạnh thật thường chiếm **nhiều pixel liên tiếp** trong M .



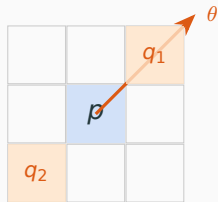
Mục tiêu Bước 3: giữ *chỉ* pixel có M cực đại cục bộ theo hướng θ .

Bước 3 - Non-maximum Suppression (NMS)

Mục tiêu: giữ **chỉ** pixel có M cực đại cục bộ dọc theo hướng gradient θ – làm mỏng cạnh còn **1 pixel**.

Thuật toán cho mỗi pixel p :

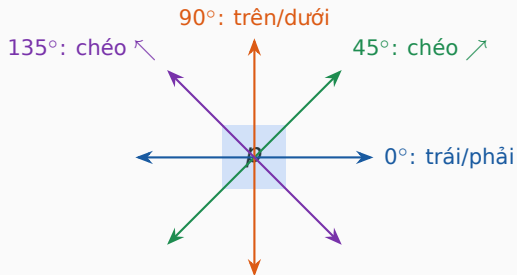
1. Xác định $\theta(p)$ – hướng gradient
2. Nội suy 2 hàng xóm q_1, q_2 dọc theo θ
3. Nếu $M(p) \geq M(q_1)$ và $M(p) \geq M(q_2)$
 $\Rightarrow$ **giữ** p
4. Ngược lại $\Rightarrow$ **xoá** p (gán 0)



so sánh $M(p)$ vs $M(q_1), M(q_2)$

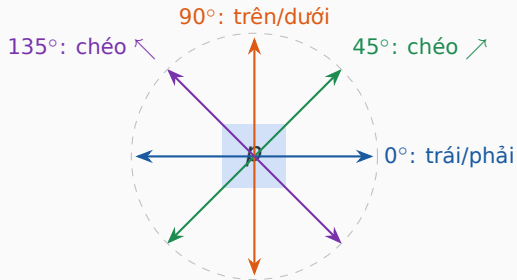
NMS - Lượng hoá hướng gradient

θ là góc liên tục; trên lưới pixel ta **lượng hoá** về 4 hướng chính.



NMS - Lượng hoá hướng gradient

θ là góc liên tục; trên lưới pixel ta **lượng hoá** về 4 hướng chính.



$\theta \in (-22.5^\circ, 22.5^\circ) \Rightarrow$ dùng hướng 0° , v.v.

NMS - Ví dụ tính tay

Cho patch M sau và $\theta(p) = 90^\circ$ (hướng thẳng đứng):

12	8 ^{$q_1=8$}	5
45	80	42
10	60 ^{$q_2=60$}	7

M (magnitude)

$\theta(p) = 90^\circ \Rightarrow$ so sánh với hàng xóm trên/dưới:

$$q_1 = M(\text{trên}) = 8$$

$$q_2 = M(\text{dưới}) = 60$$

Kiểm tra: $M(p) = 80$

$$80 \geq 8 \quad \checkmark$$

$$80 \geq 60 \quad \checkmark$$

NMS - Ví dụ tính tay

Cho patch M sau và $\theta(p) = 90^\circ$ (hướng thẳng đứng):

12	8 ^{$q_1=8$}	5
45	80	42
10	60 ^{$q_2=60$}	7

M (magnitude)

$\theta(p) = 90^\circ \Rightarrow$ so sánh với hàng xóm trên/dưới:

$$q_1 = M(\text{trên}) = 8$$

$$q_2 = M(\text{dưới}) = 60$$

Kiểm tra: $M(p) = 80$

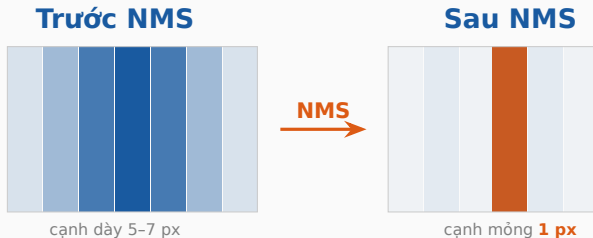
$$80 \geq 8 \quad \checkmark$$

$$80 \geq 60 \quad \checkmark$$

$\Rightarrow$ **Giữ pixel p**

Nếu $M(p) = 50$: $50 < 60 \Rightarrow$ **Xoá**

NMS - Trước và sau



Sau NMS: bản đồ cạnh mỏng, nhưng vẫn còn cạnh yếu do nhiễu. Cần

Bước 4: Hysteresis Thresholding.

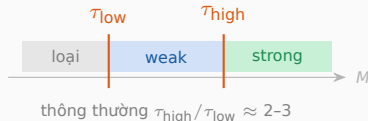
Bước 4 - Hysteresis Thresholding

Vấn đề cần giải: phân biệt **cạnh thật** với **nhiều** sau NMS.

Ý tưởng: dùng *hai* ngưỡng thay vì một.

$$\tau_{\text{low}} < \tau_{\text{high}}$$

- $M(p) > \tau_{\text{high}} \Rightarrow$ **strong edge** - giữ chắc chắn
- $\tau_{\text{low}} \leq M(p) \leq \tau_{\text{high}} \Rightarrow$ **weak edge**
- chỉ giữ nếu kết nối với strong
- $M(p) < \tau_{\text{low}} \Rightarrow$ **nhiều** - loại bỏ



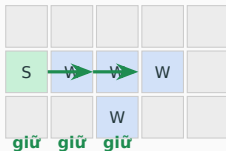
Hysteresis - Edge Tracking by Connectivity

Weak edge được giữ nếu và chỉ nếu có đường kết nối tới ít nhất một **strong edge**.



Hysteresis - Edge Tracking by Connectivity

Weak edge được giữ nếu và chỉ nếu có đường kết nối tới ít nhất một **strong edge**.

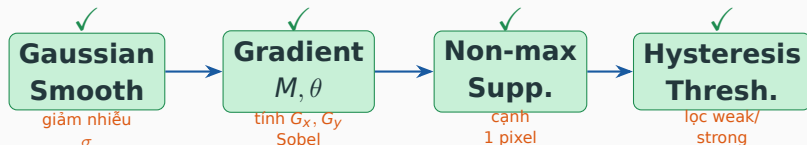


Hysteresis - Edge Tracking by Connectivity

Weak edge được giữ nếu và chỉ nếu có đường kết nối tới ít nhất một **strong edge**.



Pipeline Canny - Bức tranh toàn cảnh



3 tham số cần chỉnh:

σ - mức độ làm mờ

$\tau_{\text{low}}, \tau_{\text{high}}$ - ngưỡng lọc

Điểm khởi đầu tốt:

$\sigma = 1.0-2.0$

$\tau_{\text{high}}/\tau_{\text{low}} \approx 2-3$

Liên kết cạnh - Từ Pixel đến Contour

Sau Canny, ta có **bản đồ nhị phân** (cạnh / không cạnh).

Edge linking nhóm các pixel cạnh liên nhau thành **contour** (chuỗi điểm).

Vì sao cần contour?

- Pixel rời rạc $\neq$ đường biên có nghĩa
- Contour cho phép tính: độ dài, diện tích bao, hình dạng
- Đầu vào cho các bước tiếp theo: matching, phân đoạn

pixel map



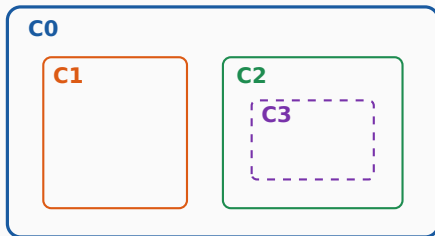
contour



OpenCV: `findContours()` trả về danh sách các contour, mỗi contour là một mảng điểm (x,y) .

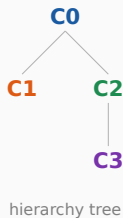
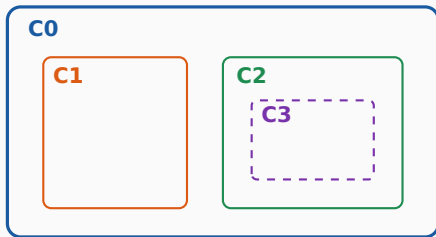
Contour Hierarchy trong OpenCV

`findContours()` trả về không chỉ contour mà còn **cấu trúc phân cấp** (hierarchy): mối quan hệ cha - con giữa các contour lồng nhau.



Contour Hierarchy trong OpenCV

`findContours()` trả về không chỉ contour mà còn **cấu trúc phân cấp** (hierarchy): mối quan hệ cha - con giữa các contour lồng nhau.



Đánh giá Chất lượng Cạnh - Precision & Recall

Để so sánh các detector, ta cần **ground truth** (cạnh đúng do người đánh nhãn).

$$\text{Precision} = \frac{TP}{TP + FP}$$

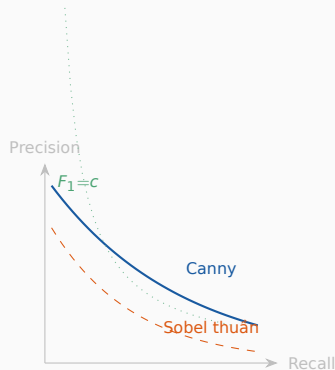
$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

TP: pixel cạnh phát hiện đúng

FP: phát hiện sai (nhiều / không phải cạnh)

FN: cạnh thật bị bỏ sót



Pitfall - Những lỗi hay gặp với Canny

- × **Lỗi 1:** τ_{high} quá thấp
⇒ nhiều weak edge được kết nối, cạnh giả xuất hiện.
- × **Lỗi 2:** τ_{high} quá cao
⇒ cạnh thật bị ngắt quãng (gap), contour không liên tục.
- × **Lỗi 3:** σ quá lớn trên ảnh có chi tiết nhỏ
⇒ cạnh gần nhau bị gộp thành một.

Hướng dẫn chỉnh tham số:

1. Bắt đầu với $\sigma = 1$,
 $\tau_{\text{high}}/\tau_{\text{low}} = 2$
2. Tăng σ nếu còn nhiễu nhiều
3. Tăng τ_{high} nếu có quá nhiều cạnh giả
4. Giảm τ_{low} nếu cạnh thật bị đứt quãng
5. Dùng Otsu auto-threshold làm baseline

So sánh các Edge Detector

Detector	Cạnh mỏng	Chống nhiễu	Tốc độ
Roberts	kém	kém	nhANH
Prewitt	trung bình	trung bình	nhANH
Sobel	trung bình	trung bình	nhANH
LoG (Marr)	tốt	tốt	trung bình
Canny	tốt nhất	tốt nhất	trung bình

LoG (Laplacian of Gaussian): làm mờ rồi lấy Laplacian; tìm cạnh tại **zero-crossing**. Tốt nhưng nhạy cảm với tham số σ .

Canny thắng vì kết hợp 4 bước bổ trợ nhau, không chỉ dùng một bộ lọc đơn.

Laplacian of Gaussian (LoG) - Marr & Hildreth

Một hướng khác để tìm cạnh: dùng **đạo hàm bậc 2** thay vì bậc 1.

Pipeline:

$$\text{LoG} = \nabla^2(G_\sigma * I) = (\nabla^2 G_\sigma) * I$$

- Làm mờ với $G_\sigma \Rightarrow$ giảm nhiễu
- Lấy Laplacian $\nabla^2 \Rightarrow$ tìm **zero-crossing**
- Zero-crossing = vị trí cạnh

Kernel LoG còn gọi là **Mexican hat** do hình dạng đặc trưng.



LoG vs. Canny - Khi nào dùng cái nào?

LoG phù hợp khi:

- Cần phát hiện **blob** và cạnh cùng lúc
- Ảnh có cấu trúc tròn (tế bào, đồng xu)
- Làm tiền xử lý cho SIFT ($\text{DoG} \approx \text{LoG}$)

Tiêu chí	LoG	Canny
Cạnh mỏng	~	✓
Chống nhiễu	✓	✓
Blob detect	✓	×
Tham số	1	3

Canny phù hợp khi:

- Cần cạnh chính xác, mỏng, liên tục
- Cần kiểm soát độ nhạy qua τ
- Làm tiền xử lý cho Hough Transform

Tự động chọn τ - Otsu Thresholding

Chỉnh τ_{low} , τ_{high} bằng tay tốn thời gian. **Otsu** tự động tìm ngưỡng tối ưu từ histogram của M .

Ý tưởng Otsu: chọn τ để **tối đa hoá phương sai liên lớp**:

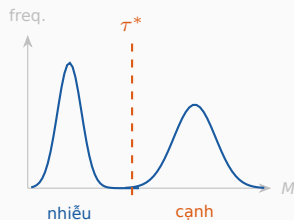
$$\tau^* = \arg \max_{\tau} \omega_0(\tau) \omega_1(\tau) [\mu_0(\tau) - \mu_1(\tau)]^2$$

ω_k : tỷ lệ pixel trong lớp k

μ_k : trung bình cường độ lớp k

Dùng trong Canny: $\tau_{\text{high}} = \tau_{\text{Otsu}}^*$,

$\tau_{\text{low}} = \tau_{\text{high}}/2$.



Canny trong Ứng dụng Thực tế

Tiền xử lý cho Hough Transform

Canny → Hough tìm đường thẳng/tròn.
Nếu bỏ Canny, Hough bị nhiễu nặng.

Phân đoạn ảnh y tế

Tìm biên tế bào, khối u trong MRI/CT.
 σ lớn để bỏ qua cấu trúc nhỏ.

Kiểm tra chất lượng (QC)

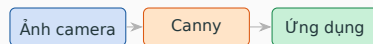
Phát hiện vết nứt, khuyết tật bề mặt
trong dây chuyền sản xuất.

Xe tự hành (ADAS)

Phát hiện vạch kẻ đường, biển báo.
Thường kết hợp Canny + RANSAC.

Document scanning

Tìm biên trang giấy để crop và
perspective-correct.



Giới hạn của Canny - Khi nào không dùng?

Canny gặp khó khi:

- **Ánh sáng không đồng đều** - cùng vật thể nhưng một bên sáng, một bên tối cho gradient khác nhau
- **Kết cấu dày đặc** (vải, cỏ) - tạo ra hàng nghìn cạnh không có nghĩa
- **Ảnh màu** - gradient trên RGB không nhất quán giữa các kênh
- **Cạnh mờ dần** (soft edge) - NMS xóa nhầm phần cạnh thật

Hướng xử lý:

- Chuẩn hoá histogram trước khi chạy Canny
- Dùng **bilateral filter** thay Gaussian để giữ cạnh sắc trong khi làm mịn vùng phẳng
- Gradient trên **Lab / HSV** thay RGB
- Kết hợp với segmentation để lọc vùng kết cấu

Từ Cạnh đến Đường thẳng

Canny cho ta **bản đồ cạnh** (pixel).

Tuần tới: làm thế nào để tìm **đường thẳng** và **đoạn thẳng** từ đó?



Từ Cạnh đến Đường thẳng

Canny cho ta **bản đồ cạnh** (pixel).

Tuần tới: làm thế nào để tìm **đường thẳng** và **đoạn thẳng** từ đó?



Phát hiện & Đối sánh Đường thẳng

Hough Transform · RANSAC · LSD · Line Matching

Xử lý ảnh và Thị giác máy tính · 121036

Từ Pixel Cảnh đến Đường thẳng

Phần trước: Canny cho ta **bản đồ cạnh** – hàng nghìn pixel.

Câu hỏi hôm nay:

**Làm thế nào để tìm ra
cấu trúc hình học *toàn cục*
ẩn sau các pixel đó?**

Ví dụ: đường kẻ đường, biên tòa nhà,
vạch kẻ sân bóng – tất cả đều là đường
thẳng.

pixel cạnh



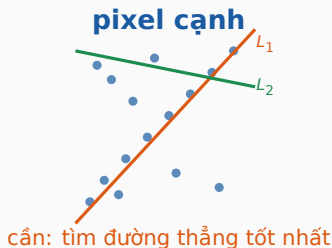
Từ Pixel Cạnh đến Đường thẳng

Phần trước: Canny cho ta **bản đồ cạnh** – hàng nghìn pixel.

Câu hỏi hôm nay:

**Làm thế nào để tìm ra
cấu trúc hình học *toàn cục*
ẩn sau các pixel đó?**

Ví dụ: đường kẻ đường, biên tòa nhà,
vạch kẻ sân bóng – tất cả đều là đường
thẳng.



Tham số hoá Đường thẳng - Vấn đề với $y = mx + b$

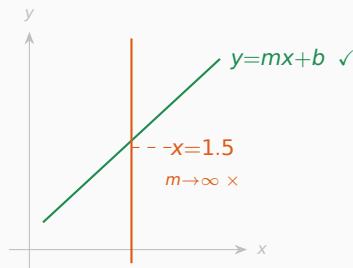
Biểu diễn quen thuộc: $y = mx + b$ (slope-intercept).

Hai tham số: m (độ dốc), b (hệ số tự do).

Vấn đề nghiêm trọng:

- Đường **thẳng đứng**: $m \rightarrow \infty$, $b \rightarrow \infty$
- Không biểu diễn được trong không gian (m, b)
- Không gian tham số **không bounded**

Cần biểu diễn khác bao phủ mọi hướng đường thẳng.

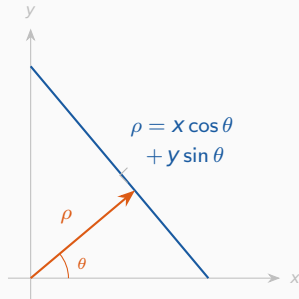


Biểu diễn (ρ, θ) - Normal Form

Giải pháp: biểu diễn đường thẳng bằng **pháp tuyến** của nó.

$$\rho = x \cos \theta + y \sin \theta$$

- ρ : khoảng cách từ **gốc tọa độ** đến đường thẳng
- θ : góc của pháp tuyến so với trục x
- $\rho \in (-d_{\max}, d_{\max})$, $\theta \in [0, 180)$
- **Mọi đường thẳng** đều có biểu diễn hữu hạn



$d_{\max} = \sqrt{W^2 + H^2}$ với ảnh $W \times H$.

Insight Cốt lõi - Điểm trong Ảnh = Đường Cong trong Accumulator

Câu hỏi ngược: cho một **điểm ảnh** (x_0, y_0) , tập hợp tất cả đường thẳng đi qua nó là gì trong không gian (ρ, θ) ?

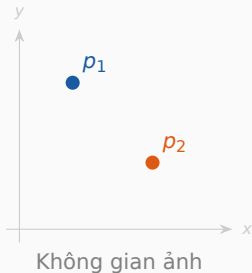
$\rho = x_0 \cos \theta + y_0 \sin \theta \implies$ một **đường cong sin** trong không gian (ρ, θ)



Insight Cốt lõi - Điểm trong Ảnh = Đường Cong trong Accumulator

Câu hỏi ngược: cho một **điểm ảnh** (x_0, y_0) , tập hợp tất cả đường thẳng đi qua nó là gì trong không gian (ρ, θ) ?

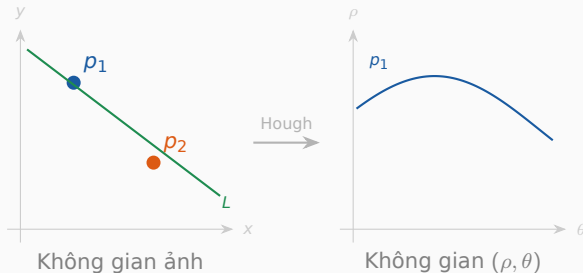
$\rho = x_0 \cos \theta + y_0 \sin \theta \implies$ một **đường cong sin** trong không gian (ρ, θ)



Insight Cốt lõi - Điểm trong Ảnh = Đường Cong trong Accumulator

Câu hỏi ngược: cho một **điểm ảnh** (x_0, y_0) , tập hợp tất cả đường thẳng đi qua nó là gì trong không gian (ρ, θ) ?

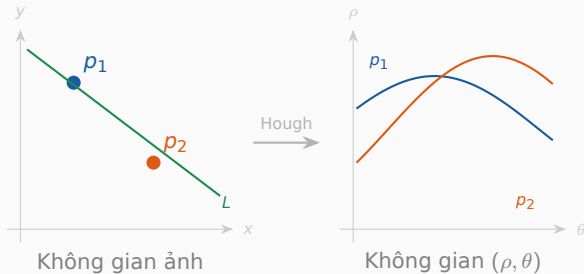
$\rho = x_0 \cos \theta + y_0 \sin \theta \implies$ một **đường cong sin** trong không gian (ρ, θ)



Insight Cốt lõi - Điểm trong Ảnh = Đường Cong trong Accumulator

Câu hỏi ngược: cho một **điểm ảnh** (x_0, y_0) , tập hợp tất cả đường thẳng đi qua nó là gì trong không gian (ρ, θ) ?

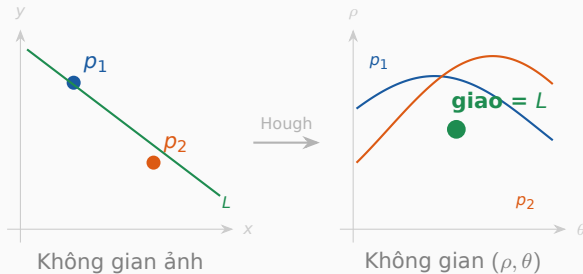
$\rho = x_0 \cos \theta + y_0 \sin \theta \implies$ một **đường cong sin** trong không gian (ρ, θ)



Insight Cốt lõi - Điểm trong Ảnh = Đường Cong trong Accumulator

Câu hỏi ngược: cho một **điểm ảnh** (x_0, y_0) , tập hợp tất cả đường thẳng đi qua nó là gì trong không gian (ρ, θ) ?

$\rho = x_0 \cos \theta + y_0 \sin \theta \implies$ một **đường cong sin** trong không gian (ρ, θ)



Accumulator - Cơ chế Bỏ phiếu

Thuật toán Hough Transform (cho đường thẳng):

1. Khởi tạo ma trận **accumulator**

$$A[\rho][\theta] = 0$$

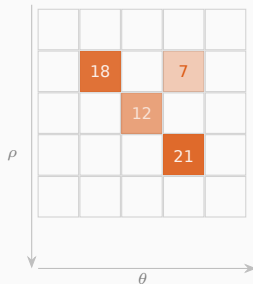
2. Với mỗi pixel cạnh (x_i, y_i) :

$$\forall \theta_k : \rho_k = x_i \cos \theta_k + y_i \sin \theta_k$$

$$A[\rho_k][\theta_k] += 1$$

3. Tìm các ô $A[\rho][\theta]$ có giá trị **lớn nhất** (local maxima)
4. Mỗi ô cực đại = một đường thẳng trong ảnh

Accumulator A



Ví dụ Tính tay - Một Pixel Bỏ phiếu

Pixel cạnh tại $(x_0, y_0) = (3, 4)$. Rời rạc hoá $\theta \in \{0, 45, 90, 135\}$:

θ	$\rho = 3 \cos \theta + 4 \sin \theta$	Ô được vote
0	$3 \cdot 1 + 4 \cdot 0 = 3.0$	$A[3.0][0]$
45	$3 \cdot 0.71 + 4 \cdot 0.71 = 4.95$	$A[5.0][45]$
90	$3 \cdot 0 + 4 \cdot 1 = 4.0$	$A[4.0][90]$
135	$3 \cdot (-0.71) + 4 \cdot 0.71 = 0.71$	$A[1.0][135]$

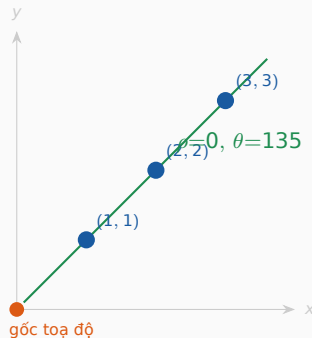
Mỗi pixel bỏ phiếu cho **tất cả** các đường thẳng có thể đi qua nó. Khi nhiều pixel cùng bỏ phiếu cho một ô (ρ^*, θ^*) , đó là dấu hiệu của một đường thẳng thật.

Ví dụ Tính tay - Ba Pixel, Một Đường thẳng

Ba pixel (1, 1), (2, 2), (3, 3) - cùng nằm trên đường $y = x$ (tức $\rho = 0$, $\theta = 135$):

Pixel	ρ tại $\theta = 135$
(1, 1)	$1(-0.71) + 1(0.71) = 0$
(2, 2)	$2(-0.71) + 2(0.71) = 0$
(3, 3)	$3(-0.71) + 3(0.71) = 0$

$\Rightarrow$ Cả 3 cùng vote vào $A[0][135]$.
 $A[0][135] = 3$ - **đỉnh cao nhất**.



Rời rạc hoá Accumulator - Độ Phân giải

Kích thước accumulator ảnh hưởng trực tiếp đến độ chính xác và tốc độ.

Tham số rời rạc hoá:

- $\Delta\theta$: bước góc (thường 1 hoặc 2)
- $\Delta\rho$: bước khoảng cách (thường 1 pixel)

$\Delta\theta$	Độ chính xác	Tốc độ
5	thấp	nhANH
1	tốt	trung bình
0.5	cao	chẬM

Kích thước accumulator:

$$N_\theta \times N_\rho = \frac{180}{\Delta\theta} \times \frac{2d_{\max}}{\Delta\rho}$$

Với ảnh 640×480 : $d_{\max} \approx 800$,
 $\Delta\theta=1$, $\Delta\rho=1$:

$$180 \times 1600 = 288\,000 \text{ ô}$$

Bộ nhớ nhỏ, tính toán song song hoá được.

HoughLines vs. HoughLinesP

OpenCV cung cấp hai biến thể:

cv2.HoughLines

Standard Hough Transform

- Output: (ρ, θ) – **đường thẳng vô hạn**
- Dùng khi cần hướng đường, không cần điểm đầu/cuối
- Không biết đoạn thẳng dài bao nhiêu



đường thẳng vô hạn

cv2.HoughLinesP

Probabilistic Hough Transform

- Output: (x_1, y_1, x_2, y_2) – **đoạn thẳng**
- Chỉ sample ngẫu nhiên subset pixel cạnh
- Nhanh hơn; có minLineLength và maxLineGap



Tham số HoughLinesP - Ý nghĩa từng cái

- **threshold**: số vote tối thiểu để công nhận đường.
Thấp → nhiều đoạn thẳng; cao → chỉ đoạn dài, rõ.
- **minLineLength**: độ dài tối thiểu (pixel).
Lọc bỏ đoạn ngắn do nhiễu.
- **maxLineGap**: khoảng hở tối đa giữa hai đoạn cùng đường để nối thành một.
Xử lý đứt quãng do Canny.



Tham số HoughLinesP - Ý nghĩa từng cái

- **threshold**: số vote tối thiểu để công nhận đường.
Thấp → nhiều đoạn thẳng; cao → chỉ đoạn dài, rõ.
- **minLineLength**: độ dài tối thiểu (pixel).
Lọc bỏ đoạn ngắn do nhiễu.
- **maxLineGap**: khoảng hở tối đa giữa hai đoạn cùng đường để nối thành một.
Xử lý đứt quãng do Canny.



Pitfall - Chọn Tham số Hough

× **threshold quá thấp:**

Hàng trăm đường thẳng giả từ nhiễu.

× **threshold quá cao:**

Bỏ sót đường thẳng ngắn hoặc bị che khuất.

× **Quên chạy Canny trước:**

Hough trực tiếp trên ảnh xám $\Rightarrow$ mọi pixel đều vote, accumulator bão hoà.

× **maxLineGap quá lớn:**

Nối các đoạn không thuộc cùng một đường.

Chiến lược chỉnh tham số:

1. Luôn **Canny trước**, Hough sau
2. Bắt đầu với $\text{threshold} = 50-100$
3. Tăng dần đến khi chỉ còn đường thẳng chính
4. $\text{minLineLength} \approx 10-20\%$ cạnh ảnh
5. $\text{maxLineGap} \approx 5-15$ pixel
6. Dùng ảnh nhỏ để thử nhanh

Độ Phức tạp & Giới hạn của Hough Transform

Độ phức tạp:

$$O(N_{\text{edge}} \times N_{\theta})$$

N_{edge} : số pixel cạnh, N_{θ} : số bước góc.

Với $N_{\text{edge}} = 10^4$, $N_{\theta} = 180$:

$\approx 1.8 \times 10^6$ phép tính - **nhANH**.

Giới hạn:

- Chỉ tìm **đường thẳng vô hạn** (HoughLines)
- Khó mở rộng sang hình phức tạp (tham số > 3)
- Nhạy với nhiễu nếu bỏ qua Canny
- Đường thẳng song song gần nhau tạo nhiều đỉnh accumulator giống nhau

⇒ Tuần tới: **RANSAC** giải quyết vấn đề nhiễu tổng quát hơn.

Mở rộng Hough - Từ Đường thẳng đến Đường tròn

Đường thẳng: 2 tham số (ρ, θ) - accumulator 2D.

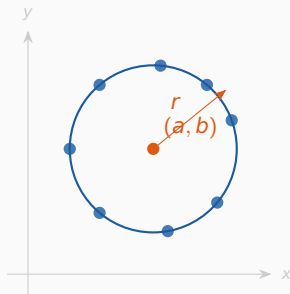
Đường tròn: **3 tham số** (a, b, r) :

$$(x - a)^2 + (y - b)^2 = r^2$$

- (a, b) : tâm đường tròn
- r : bán kính

⇒ Accumulator **3D**: $A[a][b][r]$

Bộ nhớ và thời gian tăng đáng kể,
nhưng nguyên lý bỏ phiếu giữ nguyên.



Insight - Một Điểm Ảnh trong Không gian (a, b, r)

Với điểm cạnh (x_0, y_0) , tập hợp tất cả đường tròn đi qua nó:

$$(x_0 - a)^2 + (y_0 - b)^2 = r^2 \implies \text{mặt **hình nón** trong không gian } (a, b, r)$$

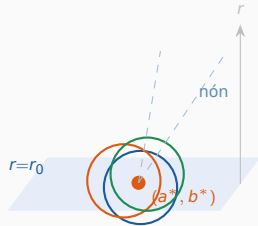
Khi cắt ngang ở $r = r_0$ cố định:

$$(x_0 - a)^2 + (y_0 - b)^2 = r_0^2$$

$\implies$ một **vòng tròn** trong mặt phẳng (a, b) .

Nhiều pixel cạnh cùng thuộc một đường tròn

$\implies$ các vòng tròn trong (a, b) **giao nhau tại (a^*, b^*)** .



$\implies$ Đỉnh accumulator = tâm đường tròn thật.

Tăng tốc - Dùng Hướng Gradient

Accumulator 3D (a, b, r) tốn bộ nhớ và thời gian.

Trick: dùng hướng gradient $\theta(x, y)$ từ Canny để giới hạn vote.

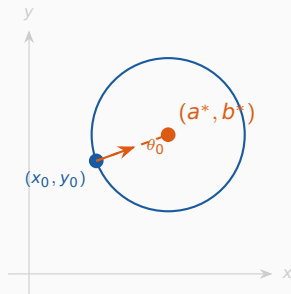
Tại pixel cạnh (x_0, y_0) với gradient hướng θ_0 :

Tâm đường tròn **chắc chắn nằm trên** đường thẳng:

$$a = x_0 + r \cos \theta_0, \quad b = y_0 + r \sin \theta_0$$

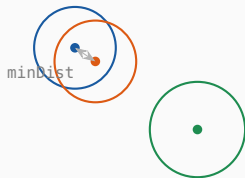
Với mỗi r , chỉ cần vote 1 ô (a, b) thay vì toàn bộ mặt phẳng (a, b) .

⇒ Giảm từ $O(N^2)$ xuống $O(N)$ vote mỗi pixel.



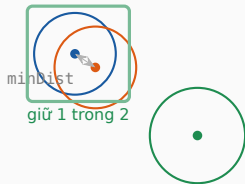
cv2.HoughCircles - Tham số Quan trọng

- **dp**: tỷ lệ độ phân giải accumulator.
dp=1: cùng kích thước ảnh; dp=2: thu nhỏ 2x.
- **minDist**: khoảng cách tối thiểu giữa hai tâm đường tròn.
Ngăn phát hiện nhiều đường tròn trùng tâm.
- **param1**: ngưỡng cao cho Canny bên trong.
- **param2**: ngưỡng vote accumulator.



cv2.HoughCircles - Tham số Quan trọng

- **dp**: tỷ lệ độ phân giải accumulator.
dp=1: cùng kích thước ảnh; dp=2: thu nhỏ 2x.
- **minDist**: khoảng cách tối thiểu giữa hai tâm đường tròn.
Ngăn phát hiện nhiều đường tròn trùng tâm.
- **param1**: ngưỡng cao cho Canny bên trong.
- **param2**: ngưỡng vote accumulator.



Giới hạn Hough - Khi nào Cần RANSAC?

Hough khó khi:

- Nhiều **outlier** không thuộc bất kỳ đường thẳng nào
⇒ nhiễu lan rộng trong accumulator
- Hình phức tạp hơn đường thẳng / tròn
⇒ tham số > 3 , không gian bùng nổ
- Nhiều đường thẳng song song gần nhau
⇒ đỉnh accumulator bị trộn lẫn
- Bán kính không biết trước
⇒ phải tìm kiếm toàn bộ r

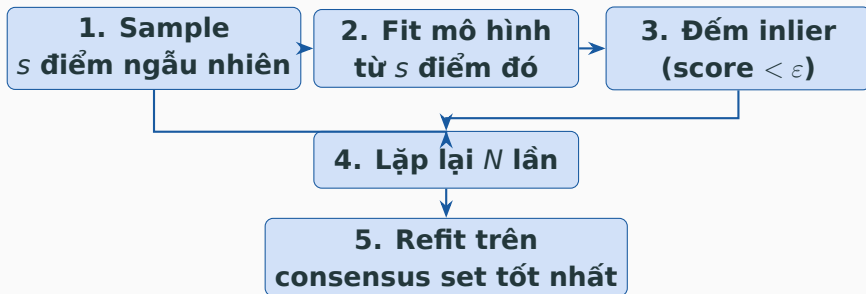
RANSAC giải quyết:

- Hoạt động tốt dù **40-60% outlier**
- Không cần rời rạc hoá tham số
- Áp dụng cho **bất kỳ mô hình nào** có thể fit từ một sample nhỏ
- Output trực tiếp là tham số mô hình, không phải accumulator

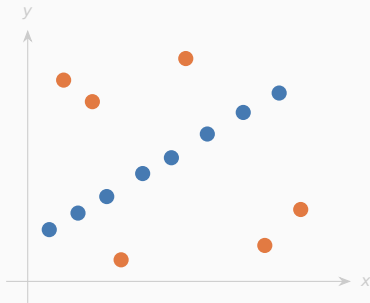
RANSAC = **RA**ndom **SA**mples **C**onsensus
Fischler & Bolles, 1981.

RANSAC - Ý tưởng Cốt lõi

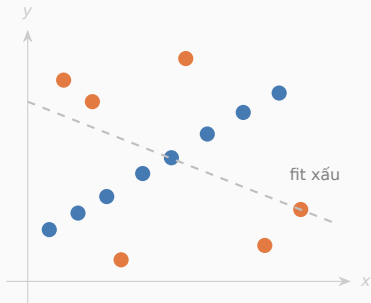
Triết lý: nếu **sample ngẫu nhiên** đủ nhỏ thì xác suất cao tất cả điểm trong sample đều là **inlier** – từ đó fit mô hình tốt.



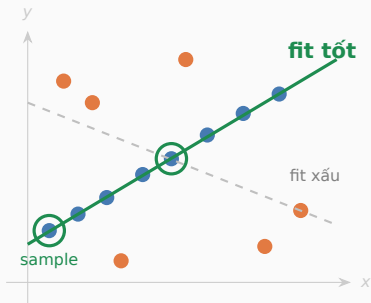
RANSAC - Iteration Theo từng Bước



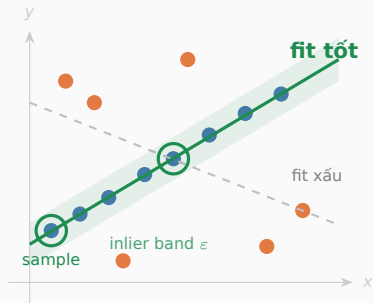
RANSAC - Iteration Theo từng Bước



RANSAC - Iteration Theo từng Bước



RANSAC - Iteration Theo từng Bước



RANSAC - Số Iteration Cần Thiết

Câu hỏi: Cần bao nhiêu iteration N để đảm bảo ít nhất một sample toàn inlier với xác suất p ?

$$N = \frac{\log(1 - p)}{\log(1 - (1 - e)^s)}$$

- e : tỷ lệ outlier (ước lượng)
- s : kích thước sample tối thiểu (đường thẳng: $s = 2$; homography: $s = 4$)
- p : xác suất thành công mong muốn
thường $p = 0.99$

e	$s = 2$	$s = 4$
10%	5	7
30%	11	26
50%	17	72
70%	55	620

số iteration N với $p = 0.99$

Với đường thẳng ($s = 2$) và 50% outlier: chỉ cần 17 iteration - **rất nhanh**.

RANSAC - Chọn Ngưỡng ϵ

ϵ (epsilon) là khoảng cách tối đa để một điểm được coi là **inlier**.

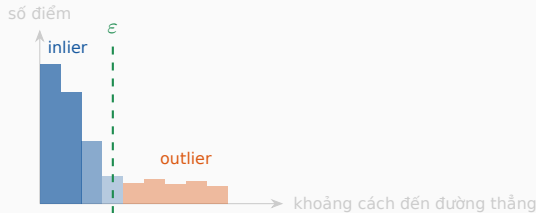
ϵ **quá nhỏ:**

- Inlier thật bị phân loại là outlier
- Consensus set nhỏ, fit không ổn định

ϵ **quá lớn:**

- Outlier bị nhận nhầm là inlier
- Fit bị kéo lệch về phía outlier

Hướng dẫn: ϵ nên phản ánh **mức nhiễu đo lường** thực tế trong dữ liệu.



RANSAC vs. Least Squares - Tại sao LS không hiệu quả

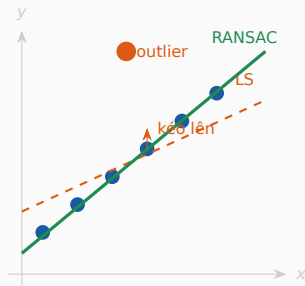
Least Squares tối thiểu hoá:

$$\sum_i (y_i - (mx_i + b))^2$$

Vấn đề: bình phương sai số khiến **outlier có ảnh hưởng rất lớn.**

Một outlier cách xa $10\times \Rightarrow$ đóng góp $100\times$ vào loss.

$\Rightarrow$ Đường fit bị **kéo mạnh** về phía outlier.



RANSAC - Ưu và Nhược điểm

Ưu điểm:

- Robust với tỷ lệ outlier cao (đến 50%+)
- Tổng quát: áp dụng cho *bất kỳ* mô hình tham số
- Không cần rời rạc hoá không gian tham số
- Dễ cài đặt; song song hoá được

Nhược điểm:

- **Không xác định**: kết quả thay đổi mỗi lần chạy
- Cần chọn ϵ và N phù hợp
- Chậm nếu e lớn và s lớn
- Chỉ tìm **một** mô hình mỗi lần chạy (dùng Multi-RANSAC / J-Linkage cho nhiều cấu trúc)
- Không hoạt động tốt khi inlier $< s$

Trong thực tế: RANSAC thường được dùng để fit **homography**, **fundamental matrix**, **pose estimation** – không chỉ đường thẳng.

Trong phần trước...

Hough tròn: accumulator 3D (a, b, r)

RANSAC: sample $\rightarrow$ fit $\rightarrow$ consensus $\rightarrow$ lặp lại

Cả hai là cơ chế **đồng thuận**

Phần này: LSD, so sánh tổng thể & ứng dụng

LSD - Line Segment Detector

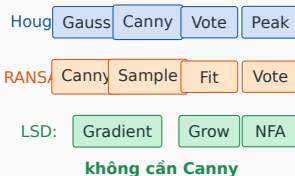
Vấn đề với Hough và RANSAC:

- Cần Canny làm tiền xử lý (tham số σ , τ)
- Hough: rời rạc hoá có thể bỏ sót đoạn thẳng ngắn
- RANSAC: chỉ tìm *một* đường mỗi lần

LSD (Grompone von Gioi et al., 2010):

- Không cần tham số chỉnh tay
- Phát hiện *tất cả* đoạn thẳng cùng lúc
- Kiểm soát số lỗi giả bằng lý thuyết xác suất
- Tuyến tính theo số pixel: $O(n)$

Pipeline so sánh

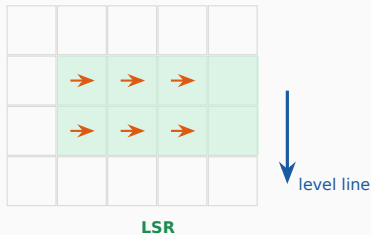


LSD - Bước 1: Level Lines & Level Region

Quan sát cốt lõi: xung quanh một đoạn thẳng, **hướng gradient** của các pixel lân cận gần như **song song** với nhau.

Định nghĩa:

- **Level line** tại pixel p : đường vuông góc với gradient $\nabla I(p)$
- **Line support region (LSR):** vùng pixel có level line cùng hướng (sai số $< \tau_\theta$, thường $\tau_\theta = 22.5$)



Mỗi LSR là **ứng viên đoạn thẳng**.
LSD kiểm tra LSR nào đủ mạnh bằng NFA.

LSD - NFA: Kiểm soát Lỗi Giả

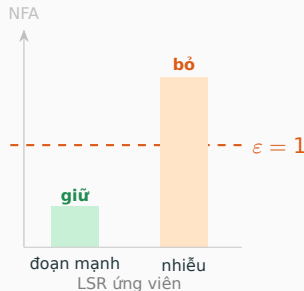
NFA (Number of False Alarms) đo xác suất một LSR xuất hiện ngẫu nhiên.

$$\text{NFA}(r) = N_T \cdot P(k \geq k_0 \mid n, p_0)$$

- N_T : tổng số LSR ứng viên cần kiểm tra
- n : số pixel trong LSR
- k_0 : số pixel có hướng gradient căn chỉnh đúng
- $p_0 = \tau_\theta / \pi$: xác suất căn chỉnh ngẫu nhiên

Quyết định:

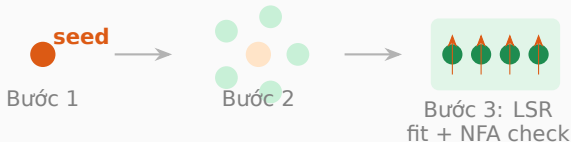
- $\text{NFA} < \varepsilon$: LSR là đoạn thẳng thật
- $\text{NFA} \geq \varepsilon$: bỏ qua



Thường dùng $\varepsilon = 1$ - kỳ vọng dưới 1 lỗi giả toàn ảnh.

LSD - Bước 2: Region Growing

Sau khi xác định seed pixel (gradient lớn nhất chưa được dùng), LSD **mở rộng vùng** bằng cách thêm pixel lân cận có hướng tương đồng.



Quá trình lặp lại cho đến khi hết pixel chưa được gán - tất cả đoạn thẳng được phát hiện **trong một lượt**.

So sánh Tổng thể - Hough / RANSAC / LSD

Tiêu chí	Hough	RANSAC	LSD
Tham số cần chỉnh	$\Delta\rho, \Delta\theta, \tau$	ε, N	không có
Cần Canny trước	có	có	không
Số đường tìm được	nhiều	một lần/lần	tất cả
Output	(ρ, θ)	tham số	đoạn thẳng (x_1, y_1, x_2, y_2)
Xử lý outlier	trung bình	tốt nhất	tốt
Tốc độ	nhanh	trung bình	tuyến tính $O(n)$
Dễ cài đặt	dễ	trung bình	phức tạp

Trong thực tế: dùng **HoughLinesP** cho ứng dụng nhanh, **RANSAC** khi nhiều

Demo 3: End-to-end trên ảnh bàn cờ numpy

1. Tạo bàn cờ 200×200 bằng numpy

2. Canny $\Rightarrow$ edge map

3. HoughLinesP với 3 bộ tham số khác nhau

4. LSD (nếu pylsd available)

5. So sánh kết quả - 4-panel figure

Ứng dụng Thực tế - Line Detection trong CV

Xe tự hành (Lane Detection):

Canny → ROI mask → HoughLinesP → gộp đường trái/phải → tính góc lái.

Phân tích tài liệu:

Phát hiện đường kẻ bảng trong hóa đơn, biểu mẫu để cắt ô tự động.

Thực tế tăng cường (AR):

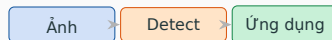
Phát hiện cạnh tường, sàn để đặt đối tượng ảo đúng phối cảnh.

Kiểm tra chất lượng:

Phát hiện vết trầy, đường nứt trên bề mặt sản phẩm - LSD cho độ chính xác cao không cần chỉnh tham số.

Đo lường 3D / Camera calibration:

Phát hiện checkerboard corners dùng line intersection. RANSAC để loại điểm nhiễu.



Pitfall - Những Lỗi Hay Gặp (Tổng hợp)

Hough:

- Bỏ qua Canny $\Rightarrow$ accumulator bão hoà
- threshold quá thấp $\Rightarrow$ đường thẳng giả
- Đường thẳng song song $\Rightarrow$ đỉnh accumulator bị trộn

RANSAC:

- ϵ quá lớn $\Rightarrow$ outlier lọt vào consensus
- N quá nhỏ $\Rightarrow$ không tìm được sample tốt
- Quên refit trên toàn bộ inlier

LSD:

- Ảnh nhiễu cao $\Rightarrow$ nhiều LSR giả dù NFA lọc
- Làm mịn nhẹ trước khi dùng LSD giúp ích
- Không có trong OpenCV core; cần `pylsd` hoặc `cv2.ximgproc`

Nguyên tắc chung:

Luôn quan sát *trực quan* kết quả trên ảnh thật trước khi tin vào con số.

Active Contours & Phân đoạn Cơ bản

Snakes · Energy Minimisation · GVF · GrabCut

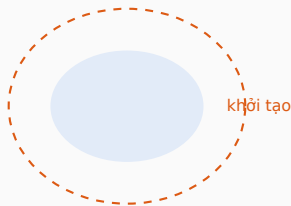
Xử lý ảnh và Thị giác máy tính · 121036

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

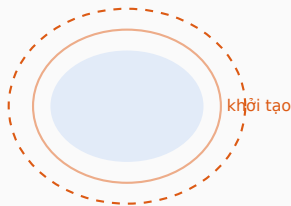
Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

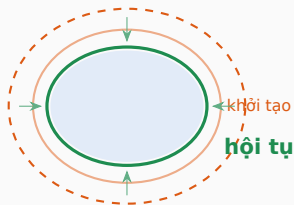
Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Hàm Năng lượng Tổng quát

Contour là tập hợp điểm $\mathbf{v}(s) = (x(s), y(s))$, $s \in [0, 1]$ (tham số cung).

Hàm năng lượng Snakes:

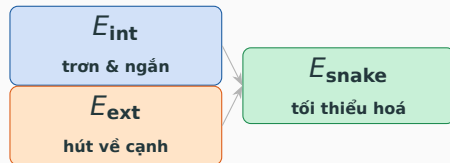
$$E_{\text{snake}} = \int_0^1 \left[\underbrace{E_{\text{int}}(\mathbf{v}(s))}_{\text{internal}} + \underbrace{E_{\text{ext}}(\mathbf{v}(s))}_{\text{external}} \right] ds$$

E_{int} : năng lượng nội tại

- kiểm soát hình dạng của contour
- ưu tiên contour trơn và ngắn

E_{ext} : năng lượng ngoại lực

- đến từ dữ liệu ảnh
- kéo contour về phía cạnh mạnh

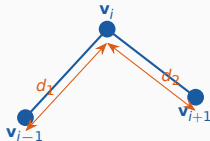


Internal Energy - Continuity Term (α)

$$E_{\text{continuity}} = \alpha \int_0^1 \left| \frac{d\mathbf{v}}{ds} \right|^2 ds \approx \alpha \sum_i |\mathbf{v}_i - \mathbf{v}_{i-1}|^2$$

Ý nghĩa vật lý: đo **tổng độ dài** contour.

- α lớn: contour muốn **co ngắn lại** như dây đàn căng – bỏ qua chi tiết
- α nhỏ: contour được phép dài hơn bám sát mọi chi tiết biên



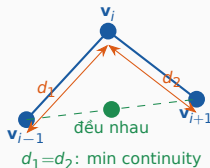
Pitfall: α quá lớn $\Rightarrow$ contour co về một điểm và biến mất.

Internal Energy - Continuity Term (α)

$$E_{\text{continuity}} = \alpha \int_0^1 \left| \frac{d\mathbf{v}}{ds} \right|^2 ds \approx \alpha \sum_i |\mathbf{v}_i - \mathbf{v}_{i-1}|^2$$

Ý nghĩa vật lý: đo **tổng độ dài** contour.

- α lớn: contour muốn **co ngắn lại** như dây đàn căng – bỏ qua chi tiết
- α nhỏ: contour được phép dài hơn bám sát mọi chi tiết biên



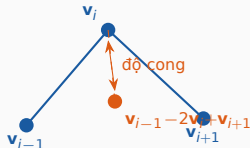
Pitfall: α quá lớn $\Rightarrow$ contour co về một điểm và biến mất.

Internal Energy - Curvature Term (β)

$$E_{\text{curvature}} = \beta \int_0^1 \left| \frac{d^2 \mathbf{v}}{ds^2} \right|^2 ds \approx \beta \sum_i |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2$$

Ý nghĩa vật lý: đo **độ cong** tại từng điểm.

- β lớn: contour muốn **thẳng hơn** như thanh kim loại cứng – không bẻ góc
- β nhỏ: contour được phép uốn cong bẻ được góc nhọn, chỗ lồi



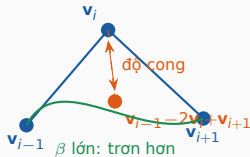
$$E_{\text{int}} = \alpha E_{\text{continuity}} + \beta E_{\text{curvature}}$$

Internal Energy - Curvature Term (β)

$$E_{\text{curvature}} = \beta \int_0^1 \left| \frac{d^2 \mathbf{v}}{ds^2} \right|^2 ds \approx \beta \sum_i |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2$$

Ý nghĩa vật lý: đo **độ cong** tại từng điểm.

- β lớn: contour muốn **thẳng hơn** như thanh kim loại cứng – không bẻ góc
- β nhỏ: contour được phép uốn cong bám được góc nhọn, chỗ lõm



$$E_{\text{int}} = \alpha E_{\text{continuity}} + \beta E_{\text{curvature}}$$

External Energy - Gradient Magnitude

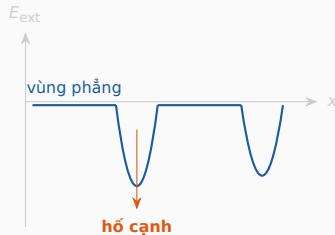
Ý tưởng: cạnh ảnh = nơi gradient lớn = **hố thế năng** hút contour.

$$E_{\text{ext}}(\mathbf{v}) = -|\nabla I(\mathbf{v})|^2$$

- Tại vùng phẳng: $|\nabla I| \approx 0 \Rightarrow E_{\text{ext}} \approx 0$ - không lực kéo
- Tại cạnh mạnh: $|\nabla I|$ lớn $\Rightarrow E_{\text{ext}}$ âm, sâu - hút mạnh

Vấn đề: vùng hút rất hẹp
- chỉ mạnh ngay tại pixel cạnh.

Contour phải khởi tạo **gần vật thể**
để "cảm nhận" được lực kéo.

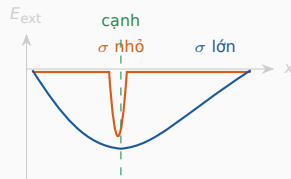


External Energy - Mở rộng Vùng Hút

Vấn đề: $-|\nabla I|^2$ hẹp - contour xa không "cảm thấy" lực.

Giải pháp: $E_{\text{ext}} = -|\nabla(G_{\sigma} * I)|^2$ - làm mờ để mở rộng vùng hút.

- σ lớn: vùng hút rộng - contour "thấy" vật thể từ xa nhưng mất localisation chính xác
- σ nhỏ: vùng hút hẹp - phải khởi tạo rất gần nhưng bám cạnh chính xác hơn



Trade-off: capture range vs. accuracy.

Giải pháp nâng cao: GVF - Chunk 3.

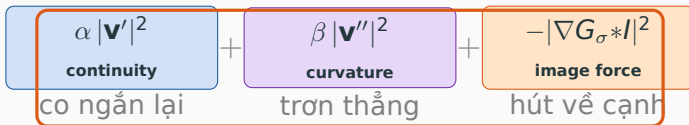
Hàm Năng lượng Đầy đủ - Ba Số hạng

$$E_{\text{snake}} = \int_0^1 \left[\underbrace{\alpha |\mathbf{v}'|^2}_{\text{continuity}} + \underbrace{\beta |\mathbf{v}''|^2}_{\text{curvature}} - \underbrace{|\nabla(G_\sigma * I)|^2}_{\text{image force}} \right] ds$$

$\alpha \mathbf{v}' ^2$ continuity	+	$\beta \mathbf{v}'' ^2$ curvature	+	$- \nabla G_\sigma * I ^2$ image force
co giãn lại		trơn thẳng		hút về cạnh

Hàm Năng lượng Đầy đủ - Ba Số hạng

$$E_{\text{snake}} = \int_0^1 \left[\underbrace{\alpha |\mathbf{v}'|^2}_{\text{continuity}} + \underbrace{\beta |\mathbf{v}''|^2}_{\text{curvature}} - \underbrace{|\nabla(G_\sigma * I)|^2}_{\text{image force}} \right] ds$$



tối thiểu hoá E_{snake} bằng gradient descent

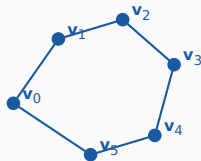
Rời rạc hoá - Từ Tích phân đến Tổng

Trên máy tính, contour là tập hữu hạn n điểm $\{\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{n-1}\}$.

Xấp xỉ đạo hàm rời rạc:

$$\mathbf{v}'_i \approx \mathbf{v}_i - \mathbf{v}_{i-1}$$

$$\mathbf{v}''_i \approx \mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}$$



$n = 6$ điểm rời rạc

Năng lượng rời rạc:

$$E \approx \sum_{i=0}^{n-1} \left[\alpha |\mathbf{v}_i - \mathbf{v}_{i-1}|^2 + \beta |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2 + E_{\text{ext}}(\mathbf{v}_i) \right]$$

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.

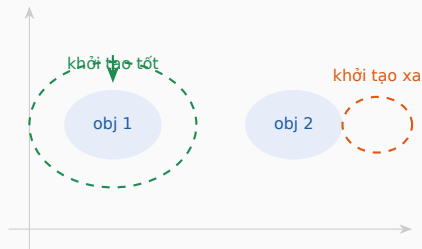


Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.

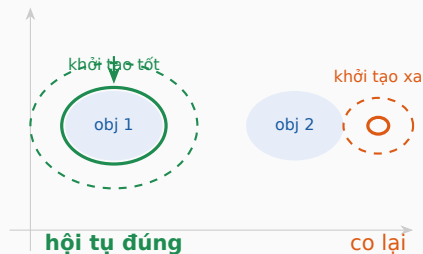


Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.



Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Demo 1 - External Energy Landscape

Demo 1: Visualise $E_{\text{ext}} = -|\nabla I|^2$

1. Tạo ảnh đĩa tròn 150×150
(numpy)

2. Gaussian blur $\Rightarrow$ tính $|\nabla I|^2$

3. $E_{\text{ext}} = -|\nabla I|^2$

4. imshow heatmap + đường
đẳng mức

5. Overlay contour khởi tạo
và vị trí lý tưởng

Kiểm tra Nhận thức

α kiểm soát điều gì trong hàm năng lượng Snakes?

Kiểm tra Nhận thức

α kiểm soát điều gì trong hàm năng lượng Snakes?

Continuity - độ dài contour; α lớn khiến contour co ngắn lại.

Kiểm tra Nhận thức

α kiểm soát điều gì trong hàm năng lượng Snakes?

Continuity - độ dài contour; α lớn khiến contour co ngắn lại.

Tại sao cần làm mờ ảnh trước khi tính E_{ext} ?

Kiểm tra Nhận thức

α kiểm soát điều gì trong hàm năng lượng Snakes?

Continuity - độ dài contour; α lớn khiến contour co ngắn lại.

Tại sao cần làm mờ ảnh trước khi tính E_{ext} ?

Mở rộng vùng hút - contour khởi tạo xa vẫn "cảm thấy" lực kéo về cạnh.

Kiểm tra Nhận thức

α kiểm soát điều gì trong hàm năng lượng Snakes?

Continuity - độ dài contour; α lớn khiến contour co ngắn lại.

Tại sao cần làm mờ ảnh trước khi tính E_{ext} ?

Mở rộng vùng hút - contour khởi tạo xa vẫn "cảm thấy" lực kéo về cạnh.

Snakes là bài toán thuộc loại nào? Tìm kiếm hay tối ưu hoá?

α kiểm soát điều gì trong hàm năng lượng Snakes?

Continuity - độ dài contour; α lớn khiến contour co ngắn lại.

Tại sao cần làm mờ ảnh trước khi tính E_{ext} ?

Mở rộng vùng hút - contour khởi tạo xa vẫn "cảm thấy" lực kéo về cạnh.

Snakes là bài toán thuộc loại nào? Tìm kiếm hay tối ưu hoá?

Tối ưu hoá - tìm contour $\mathbf{v}(s)$ tối thiểu hoá E_{snake} .

Ý tưởng cốt lõi - Chunk 1

Contour = dây cao su: muốn **ngắn** và **trơn**, đồng thời bị **hút về cạnh**.

Trạng thái cân bằng = cực tiểu của E_{snake} .

$$E_{\text{snake}} = \int_0^1 \underbrace{[\alpha |\mathbf{v}'|^2 + \beta |\mathbf{v}''|^2]}_{E_{\text{int}}} \underbrace{- |\nabla G_{\sigma} * I|^2}_{E_{\text{ext}}} ds$$

tối thiểu hoá bằng gradient descent rời rạc

Trong phần trước...

$$E_{\text{int}} = \alpha |\mathbf{v}'|^2 + \beta |\mathbf{v}''|^2 - \text{trơn và ngăn}$$

$$E_{\text{ext}} = -|\nabla(G_\sigma * I)|^2 - \text{hút về cạnh}$$

Rời rạc hoá: n điểm $\mathbf{v}_i$, đạo hàm sai phân

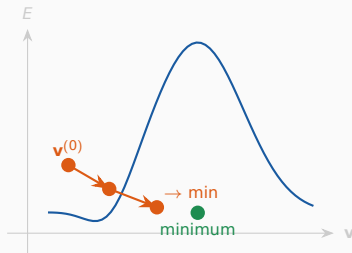
Phần này: tối ưu hoá, ma trận A , trade-off tham số

Tối ưu hoá - Gradient Descent Rời rạc

Mỗi điểm $\mathbf{v}_i = (x_i, y_i)$ được cập nhật theo hướng giảm gradient của E_{snake} :

$$\mathbf{v}_i^{(t+1)} = \mathbf{v}_i^{(t)} - \gamma \left. \frac{\partial E_{\text{snake}}}{\partial \mathbf{v}_i} \right|^{(t)}$$

- γ : **bước học** (step size / time step)
- Tất cả n điểm được cập nhật **đồng thời** mỗi iteration
- Lặp đến khi hội tụ
 $\|\mathbf{v}^{(t+1)} - \mathbf{v}^{(t)}\| < \delta$



Pitfall: γ quá lớn

$\Rightarrow$ contour dao động, không hội tụ.

Ma trận Hệ số A - Implicit Scheme

Gradient của E_{int} theo $\mathbf{v}$ có dạng **tuyến tính**:

$$\frac{\partial E_{\text{int}}}{\partial \mathbf{v}} = A \mathbf{v}$$

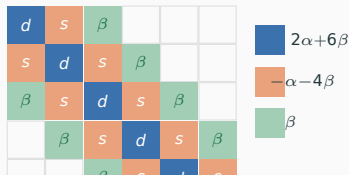
A là ma trận **pentadiagonal** (5 đường chéo) kích thước $n \times n$:

- Đường chéo chính: $2\alpha + 6\beta$
- ± 1 bên: $-\alpha - 4\beta$
- ± 2 bên: β

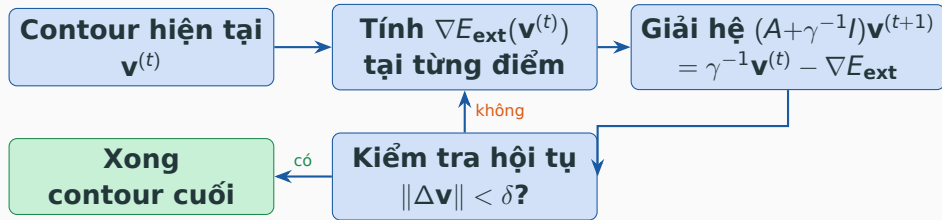
Cập nhật **implicit** (ổn định hơn explicit):

$$(A + \gamma^{-1}I) \mathbf{v}^{(t+1)} = \gamma^{-1} \mathbf{v}^{(t)} - \nabla E_{\text{ext}}$$

A (pentadiagonal)



Một Iteration Hoàn chỉnh



$\nabla E_{\text{ext}}(\mathbf{v}_i) = -2 \nabla \left[|\nabla(G_\sigma * I)|^2 \right]_{\mathbf{v}_i}$ - nội suy bằng bilinear interpolation tại vị trí contour.

Tham số α - Continuity Weight

α nhỏ: bám sát

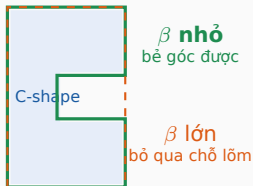


α lớn: co, bỏ chi tiết

α	Hiệu ứng	Quy
≈ 0.01	Bám sát biên; nhạy cảm với nhiễu cục bộ	
≈ 0.1	Cân bằng; hầu hết các ứng dụng	
≈ 1.0	Contour mượt; bỏ qua chi tiết biên nhỏ; có thể co biến mất	

tắc thực tế: bắt đầu với $\alpha = 0.01$; tăng nếu contour dao động hoặc bị nhiễu kéo.

Tham số β - Curvature Weight



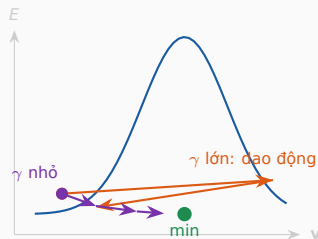
β	Hiệu ứng	Quy
≈ 0.0	Cho phép góc nhọn; dễ bị nhiễu cục bộ	
≈ 0.1	Trơn vừa; chuẩn cho hầu hết ảnh y tế	
≈ 1.0	Contour rất cứng; không thể vào chỗ lõm	

tắc thực tế: $\beta = 0.1$ là điểm khởi đầu tốt; giảm nếu cần bám biên góc nhọn hoặc chỗ lõm.

Tham số γ - Step Size

γ kiểm soát tốc độ hội tụ và ổn định số học.

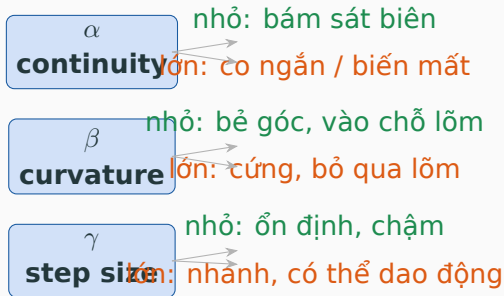
- γ lớn: ít iteration hơn, nhưng có thể **dao động** qua cực tiểu
- γ nhỏ: ổn định, chính xác, nhưng **chậm** - cần nhiều iteration
- Scheme implicit cho phép γ lớn hơn
scheme explicit mà vẫn ổn định



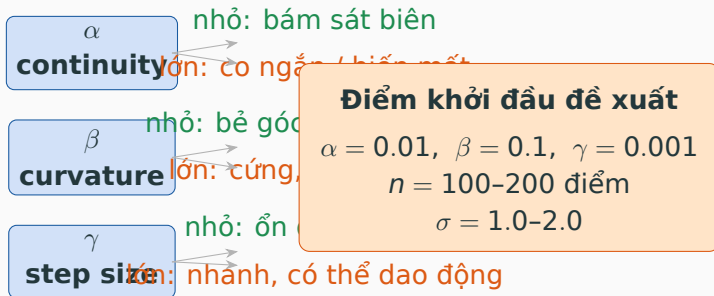
Trong `skimage.active_contour`:

tham số `max_num_iter` (số vòng lặp) và `convergence` (ngưỡng dừng) thay cho γ trực tiếp.

Ba Tham số - Tổng hợp Trade-off



Ba Tham số - Tổng hợp Trade-off



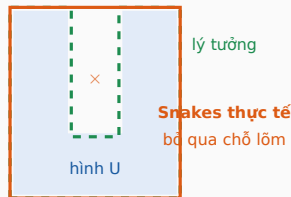
Vấn đề: Concavity Problem

Snakes cố định **không thể** lọt vào vùng lõm sâu.

Tại sao?

Lực E_{ext} tại vùng lõm nhỏ hoặc hướng ra ngoài.

Lực E_{int} (continuity) kéo contour **thẳng qua** chỗ lõm.



Biểu hiện:

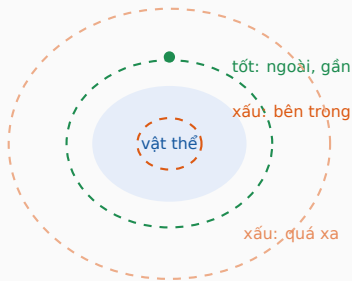
- Biên chữ U, chữ C, hình sao
- Contour "cắt qua" chỗ lõm thay vì đi vào
- Tăng β làm vấn đề trầm trọng hơn

Giải pháp: GVF – sẽ nói ở Chunk 3

Vấn đề: Nhạy với Khởi tạo - Chi tiết

Ba quy tắc thực tế để khởi tạo contour hiệu quả:

1. **Bao quanh** vật thể, không cắt vào trong.
Contour bên trong $\rightarrow$ lực E_{ext} có thể đẩy ra ngoài thay vì co vào.
2. Khoảng cách đến biên $\leq 3\sigma$
Ngoài capture range của $E_{\text{ext}} \rightarrow$ contour co lại không tìm được cạnh.
3. Số điểm n đủ dày
Quá ít điểm $\rightarrow$ đoạn thẳng dài giữa hai điểm; curvature term thống trị.



API chính:

- `active_contour(image, snake, ...)`
- `image`: ảnh float, đã Gaussian blur
- `snake`: array $(n, 2)$ các điểm (y, x) khởi tạo
- `alpha, beta, gamma`: 3 tham số chính
- `max_num_iter`: số vòng lặp tối đa
- `convergence`: ngưỡng dừng sớm

Output: array $(n, 2)$ – vị trí cuối của contour

```
from skimage.filters import gauss
from skimage.segmentation \
    import active_contour
```

```
img_blur = gaussian(img, sigma=2)
```

```
# Khởi tạo hình tròn
t = np.linspace(0, 2*np.pi, 200)
init = np.c_[
    cy + r*np.sin(t),
    cx + r*np.cos(t)
]
```

```
snake = active_contour(
    img_blur, init,
```

Demo 2 - active_contour với skimage

Demo 2: Quan sát ảnh hưởng trực tiếp của α và β

1. `skimage.data.coins()` - crop một đồng xu
2. Khởi tạo contour tròn bên ngoài
3. Chạy 3 bộ (α, β) khác nhau
4. Plot 3 kết quả cạnh nhau
5. Giải thích tại sao mỗi bộ cho kết quả khác

Bộ tham số: (0.01, 0.1) - tight fit | (0.1, 0.1) - over-smooth | (0.01, 1.0) - stiff

Kiểm tra Nhận thức

Ma trận A trong Snakes có dạng gì? Tại sao điều đó có lợi?

Kiểm tra Nhận thức

Ma trận A trong Snakes có
dạng gì? Tại sao điều đó có lợi?

Pentadiagonal (5 đường chéo) $\Rightarrow$
giải hệ tuyến tính $O(n)$ thay vì $O(n^3)$.

Kiểm tra Nhận thức

Ma trận A trong Snakes có dạng gì? Tại sao điều đó có lợi?

Pentadiagonal (5 đường chéo) $\Rightarrow$ giải hệ tuyến tính $O(n)$ thay vì $O(n^3)$.

Tăng β giải quyết hay làm trầm trọng hơn concavity problem?

Kiểm tra Nhận thức

Ma trận A trong Snakes có dạng gì? Tại sao điều đó có lợi?

Pentadiagonal (5 đường chéo) $\Rightarrow$ giải hệ tuyến tính $O(n)$ thay vì $O(n^3)$.

Tăng β giải quyết hay làm trầm trọng hơn concavity problem?

Làm trầm trọng hơn - β lớn = contour cứng hơn, càng khó bẻ vào chỗ lồi.

Kiểm tra Nhận thức

Ma trận A trong Snakes có dạng gì? Tại sao điều đó có lợi?

Pentadiagonal (5 đường chéo) $\Rightarrow$ giải hệ tuyến tính $O(n)$ thay vì $O(n^3)$.

Tăng β giải quyết hay làm trầm trọng hơn concavity problem?

Làm trầm trọng hơn - β lớn = contour cứng hơn, càng khó bẻ vào chỗ lồi.

Contour khởi tạo bên trong vật thể - vấn đề gì xảy ra?

Ma trận A trong Snakes có dạng gì? Tại sao điều đó có lợi?

Pentadiagonal (5 đường chéo) $\Rightarrow$ giải hệ tuyến tính $O(n)$ thay vì $O(n^3)$.

Tăng β giải quyết hay làm trầm trọng hơn concavity problem?

Làm trầm trọng hơn - β lớn = contour cứng hơn, càng khó bẻ vào chỗ lồi.

Contour khởi tạo bên trong vật thể - vấn đề gì xảy ra?

Lực E_{ext} có thể đẩy ra ngoài; continuity co về tâm $\Rightarrow$ co biến mất.

Ý tưởng cốt lõi - Chunk 2

A pentadiagonal + implicit scheme $\Rightarrow$ giải $O(n)$ mỗi iteration.

α : co/dài - β : cứng/mềm - γ : nhanh/ổn định.

Hai giới hạn chính: nhạy với khởi tạo & không vào được chỗ lõm.

$$(A + \gamma^{-1}I) \mathbf{v}^{(t+1)} = \gamma^{-1} \mathbf{v}^{(t)} - \nabla E_{\text{ext}}$$

Chunk 3: GVF giải concavity, GrabCut cầu nối sang region-based